8-Puzzle Solver Using Artificial Intelligence

A Comparative Heuristic Shootout using the A* Search Algorithm

1. Title Page Details

Project Title	8-Puzzle Solver using Artificial Intelligence (Heuristic A* Search)
Course	Artificial Intelligence
Student Name	[name]
Roll Number	[roll no]
College	[college]
Project Guide	[professor name]
Language & Paradigm	Python 3 (Informed State-Space Search)
Algorithm & Heuristics	A* Search: Misplaced Tiles, Manhattan Distance, Enhanced Manhattan (Linear Conflict)
Output Interface	Command-Line Interface (Console)

2. Abstract

The 8-puzzle is a classic AI state-space search problem consisting of eight numbered sliding tiles and one blank space arranged on a 3x3 grid. The objective is to reach a designated target arrangement from an arbitrary start state in the minimum number of sliding moves. This project implements an intelligent solver in Python utilizing the A* Search algorithm to empirically compare three heuristic functions: Misplaced Tiles, Manhattan Distance, and an Enhanced Manhattan heuristic incorporating Linear Conflicts. Mathematical solvability is verified before search via inversion parity to prevent infinite loops. Experimental findings confirm that while all three heuristics find the optimal path, the Enhanced Manhattan heuristic achieves superior search pruning, cutting node expansions by up to 83.6%.

3. Problem Statement

The 8-puzzle consists of a 3x3 grid with eight numbered tiles (1 to 8) and a blank space (0). Tiles adjacent to the blank space can slide horizontally or vertically into the empty space. Given an arbitrary configuration, the system must: (1) verify whether the initial state is mathematically solvable toward GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0); (2) compute the optimal (shortest) sequence of legal moves; and (3) quantify and benchmark the search efficiency of different heuristic functions.

4. Objectives

- Formulate the 8-puzzle as a formal Artificial Intelligence state-space search graph.
- Implement an inversion-parity mathematical check (is_solvable) to detect unsolvable states in O(N^2) time.
- Implement the A* Search algorithm using Python's heapq priority queue and back-pointer path reconstruction.
- Code and benchmark three distinct admissible heuristics: Misplaced Tiles (h1), Manhattan Distance (h2), and Enhanced Manhattan with Linear Conflict (h3).
- Measure, compare, and display solution moves, nodes expanded, and nodes discovered in an interactive console.

5. Tools and Technologies

Component	Technology	Purpose
Language	Python 3.x	Core logic, state representation, and heuristics implementation.
Algorithm	A* Search Algorithm	Informed best-first graph search optimizing f(n) = g(n) + h(n).
Data Structure	heapq (Min-Heap)	Binary min-heap providing O(log V) priority queue retrieval.
Tie Breaker	itertools.count	Provides unique sequential IDs to resolve priority queue ties cleanly.
Interface	Console / CLI	Formatted ASCII board display, user input prompt, and comparison table.

6. Algorithm Explanation with Pseudocode

A* is an informed search algorithm that navigates a state graph by evaluating candidate states using **f(n) = g(n) + h(n)**, where **g(n)** is the exact cost from start to state n, and **h(n)** is the estimated cost from state n to the goal. A priority queue maintains nodes ordered by lowest f(n). When current == GOAL, the optimal path is reconstructed using the parent pointer mapping.

```
Algorithm: AStar(start, heuristic)
1. Initialize queue as PriorityQueue
2. Push (heuristic(start), 0, tie_counter, start) into queue
3. best_g[start] = 0; parent[start] = (None, None); expanded = 0
4. WHILE queue is not empty DO
5.     Pop (f, g, _, current) from queue
6.     IF g != best_g[current] THEN CONTINUE // skip stale entry
7.     IF current == GOAL THEN RETURN ReconstructPath(parent, current)
8.     expanded = expanded + 1
9.     FOR EACH (neighbor, move) in GetNeighbors(current) DO
10.        new_g = g + 1
11.        IF new_g < best_g.get(neighbor, infinity) THEN
12.            best_g[neighbor] = new_g; parent[neighbor] = (current, move)
13.            Push (new_g + heuristic(neighbor), new_g, next_tie, neighbor) into queue
14. RETURN Failure
```

7. State Representation and Conflict-Checking Logic

State Representation: The board is represented as an immutable 9-element tuple: state = (s0, s1, ..., s8). Grid coordinates are computed via row = i // 3, col = i % 3. GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0).
Solvability & Inversion Parity: An inversion occurs when a larger tile precedes a smaller tile in row-major order (ignoring 0). Horizontal sliding changes 0 inversions; vertical sliding changes inversions by 0, +2, or -2. Inversion parity is invariant. Since GOAL has 0 inversions (even), any solvable start state must have an even number of inversions.
Heuristic Logic:
1. *Misplaced Tiles (h1):* Counts non-zero tiles not in GOAL[i]. Admissible because each misplaced tile moves at least once.
2. *Manhattan Distance (h2):* Sum of abs(row - goal_row) + abs(col - goal_col). Admissible; dominates h1.
3. *Enhanced Manhattan (h3):* Manhattan distance + 2 * (disjoint linear conflicts). A linear conflict occurs when two tiles in the same row/col have their goal positions reversed. Resolving it requires at least 2 extra moves. Disjoint conflicts preserve admissibility while dominating h2.

8. Code Explanation (Function by Function)

- **misplaced_tiles(state):** Iterates through state with enumerate and sums 1 whenever tile != 0 and tile != GOAL[i].
- **manhattan_distance(state):** For each tile != 0, computes (row, col) = divmod(i, 3), looks up (goal_row, goal_col) in GOAL_POS, and sums absolute coordinate differences.
- **enhanced_manhattan(state):** Starts with Manhattan distance. Scans all rows and columns for tiles belonging in that line whose relative goal positions are inverted. Uses a greedy matching loop with a 'used' set to count non-overlapping conflicting pairs and returns distance + 2 * disjoint_count.
- **is_solvable(state):** Extracts non-zero numbers, counts all inverted pairs (i < j and values[i] > values[j]), and returns inversions % 2 == 0.

- **get_neighbors(state):** Finds blank index via state.index(0), computes (row, col), tests moves in 4 directions [Up, Down, Left, Right], bounds-checks coordinates, swaps the blank with the neighbor tile, and yields (new_state, move).
- **astar(start, heuristic):** Main search loop using heapq. Uses itertools.count as a tie-breaker. Pops min-f state, checks if stale via best_g, checks for GOAL, and expands neighbors. Returns (path, expanded, discovered).
- **print_board(state):** Slices the 9-element tuple into 3 lines of 3 characters, displaying 0 as an underscore '_'.
- **main():** Coordinates execution: prompts user for 9 digits (or Enter for sample), validates length, permutation, and solvability, benchmarks all 3 heuristics, and displays comparison table and step-by-step solution path.

9. How to Run

1. Open terminal in project folder: cd e:\8_Queen_Solver
2. Run: py proj.py (or python proj.py)
3. Press **Enter** to run the built-in sample puzzle (1 2 3 4 0 6 7 5 8) or type 9 numbers.

10. Sample Output

```
================================================
       8-PUZZLE HEURISTIC SHOOTOUT
================================================
GOAL STATE:
1 2 3
4 5 6
7 8 _

PERFORMANCE COMPARISON (Puzzle: 2 1 3 4 5 6 8 7 0)
--------------------------------------------------------------------------
Heuristic                    Moves     Expanded   Discovered
--------------------------------------------------------------------------
Misplaced Tiles                 22         9055        13885
Manhattan Distance              22         2375         3683
Enhanced Manhattan              22         1487         2329
--------------------------------------------------------------------------
SOLUTION PATH: Enhanced Manhattan (22 steps from start to goal)
```

11. Time and Space Complexity Analysis

A* Time Complexity: O(b^d) worst case, where b is the branching factor (2 to 3) and d is solution depth. A stronger heuristic reduces the effective branching factor b*, shrinking the search tree dramatically.
A* Space Complexity: O(b^d) because A* retains all discovered states in best_g and the priority queue.
Comparison with Uninformed Search (BFS): For depth 22, BFS generates >100,000 states blindly. A* with Enhanced Manhattan explores only 1,487 states (over 98% reduction), while guaranteeing identical optimal path length.

12. Results (Performance Shootout)

Heuristic Evaluated	Moves	Expanded Nodes	Discovered Nodes	Search Reduction
Misplaced Tiles (h1)	22	9,055	13,885	Baseline (0%)
Manhattan Distance (h2)	22	2,375	3,683	73.8% fewer nodes
Enhanced Manhattan (h3)	22	**1,487**	**2,329**	**83.6% fewer nodes**

13. Limitations

- **Exponential Memory Scaling:** Retaining all visited states in best_g and heapq makes standard A* impractical for 15-puzzle (4x4) or 24-puzzle (5x5) without memory bounds.
- **Redundant Calculation in Heuristic:** Lines 46-79 calculate a raw conflict count that is unused, while lines 82-127 recompute pairs for greedy selection.
- **Console Only:** The interface is text-only without mouse interaction or animations.

14. Future Scope

- **Memory-Bounded Algorithms:** Implement IDA* (Iterative Deepening A*) or SMA* (Simplified Memory-Bounded A*) to solve larger grid sizes with O(d) memory.
- **Pattern Databases:** Precompute disjoint additive pattern databases to create even stronger heuristics.
- **Graphical Interface:** Develop a Tkinter/Pygame GUI with animated tile transitions and real-time step playback.

15. Conclusion

This project successfully implemented and benchmarked an intelligent 8-puzzle solver using the A* Search algorithm in Python. Pre-checking solvability via inversion parity guarantees that impossible boards are rejected immediately. The heuristic shootout empirically validated the theory of heuristic dominance: adding disjoint linear conflicts to Manhattan Distance cuts node expansions by 83.6% compared to Misplaced Tiles while preserving 100% path optimality (22 moves).

16. References

1. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Hansson, O., Mayer, A., & Kulzer, W. (1992). *Heuristic-Search Approaches to Board Games: Linear Conflict*. Advances in Computer Chess, 7.
3. Korf, R. E., & Taylor, L. A. (1996). *Finding optimal solutions to the twenty-four puzzle*. AAAI Proceedings.
4. Python Software Foundation. *heapq - Heap queue algorithm*. Python 3 Documentation.

17. Viva-Voce Questions and Answers

Q1: What conditions must a heuristic satisfy for A* to guarantee an optimal solution?

A heuristic h(n) must be admissible (it never overestimates the true cost to reach the goal, h(n) <= h*(n)). For graph search where closed states are not reopened, it should also be consistent (monotonic), satisfying h(n) <= c(n, a, n') + h(n').

Q2: What is 'Heuristic Dominance' and which heuristic dominates in our project?

If ha(n) >= hb(n) for all states n, and both are admissible, then ha dominates hb. A dominating heuristic will never expand more nodes than a dominated one. In our project, Enhanced Manhattan (h3) dominates Manhattan Distance (h2), which dominates Misplaced Tiles (h1).

Q3: How does the solvability test work using inversion counting?

An inversion is any pair of numbered tiles where a larger number precedes a smaller number in row-major order (ignoring 0). Sliding horizontally changes 0 inversions; sliding vertically changes inversions by 0, +2, or -2. Parity is invariant. Because the goal state has 0 inversions (even), any solvable start state must have an even number of inversions.

Q4: What is a 'Linear Conflict' and why does it add 2 moves?

A linear conflict occurs when two tiles belong in their goal line (row or column) but are in reverse relative order. One tile must move out of the line into an adjacent line and then move back to let the other tile pass, adding at least 2 moves (1 step away + 1 return) not counted in standard Manhattan distance.

Q5: Why did we use itertools.count() in the priority queue tuple?

In Python's heapq, tuples are compared element-by-element: (f, g, tie, state). If two states have identical f and g values, Python compares the third element. next(tie) provides a strictly incrementing integer that resolves ties immediately, preventing Python from attempting to compare board tuples.